

Project Specification

1. Project Overview

The goal of this project is to build a high-performance scheduler for the two-party **SHARK** [SHARK: Actively Secure Inference Using Function Secret Sharing](#) Secure Neural Network Inference (SNNI) protocol. [kanav99/shark: A framework for actively secure inference](#) SHARK is a bi-phase protocol (Preprocessing by dealer and Inference by client and server). The current implementation is a "one-shot" and limited to single-execution. This scheduler will transform SHARK into a multi-user, concurrent service capable of handling multiple models listed in github repo benchmarking folder (HiNet, AlexNet, VGG16, M1, M2, D4, ResNet50) while meeting strict Service Level Objectives (SLOs).

2. Technical Context & Scope

2.1 The Execution Engine (SHARK)

The developer will work with the existing SHARK C++ binaries.

- **Preprocessing (The Dealer):** Generates `server.dat` and `client.dat` keys.
- **Inference (Server/Client):** Uses the keys to perform secure computation over TCP.
- **The Limitation:** Pre-processed Files are one-time use, and should not be reused, and names are hardcoded.
- **The Fix:** The scheduler must modify the execution calls to pass: (these are simple fixes a couple of lines and I can guide, if required may provide code as well)
 1. **Unique Filenames:** Using timestamps (e.g., `hinet_batch4_1704.dat`) to allow concurrent runs.
 2. **Specific Ports:** Assigning a unique port per inference session.
 3. **Batch Sizes:** Support for variable batching (1, 2, 4, 8).

2.2 Proactive Preprocessing (The File Pool)

The scheduler must maintain a "Buffer" of pre-computed keys.

- **Differential Buffering:** Lightweight models (AlexNet) may have a buffer of 3, while heavy models (VGG16) may be restricted to 1.
- **Background Maintenance:** A dedicated thread must proactively replenish these files so that when a request arrives, the "Inference" phase can start immediately.

3. System Architecture & Workflow

3.1 The Request Lifecycle

1. **Client Request:** Client sends a request: `ModelName`, `BatchSize`, and SLO (e.g., "Finish in 500 milliseconds").
2. **Admission Control:** The scheduler checks the current system load and historical data.
 - *Accept:* If it is likely to meet the SLO.
 - *Reject:* If the system is too congested to meet the deadline.
3. **Job Queueing:** Accepted jobs enter a queue to be sorted by one of the enabled scheduling algorithms.
4. **Dispatching:** When CPU cores and a "Ready" pre-computed file are available, the scheduler spawns a SHARK server instance on a unique port and designated CPU ports.
5. **Execution:** The Scheduler informs the client of the `IP` and `Port`. The client then runs its one-shot SHARK client to complete the task.
6. **Cleanup:** Once the process exits, the unique files are deleted, and resources (cores/ports) are returned to the pool.

4. Scheduling Methodologies

The developer must implement a "Plug-and-Play" interface to switch between these algorithms:

4.1 Baselines (Control Group)

- **First-Come-First-Served (FCFS):** Execute in order of arrival.
- **Shortest Processing Time (SPT) with Aging:** Run the fastest jobs first to maximize throughput.

4.2 SLO-Aware Heuristics

- **Earliest Deadline First (EDF):** Prioritize jobs with the nearest absolute deadline.
- **Least Slack Time (LST):** Calculate "Slack" (Time remaining until deadline - Time required to run). Prioritize jobs with the lowest slack.

4.3 Computational Intelligence

- **Simulated Annealing (SA):** Periodically re-orders the queue to find a sequence that minimizes the total number of missed deadlines.
- **Deep Q-Network (DQN):** Use a pre-trained or online Reinforcement Learning agent to select the next job based on queue state and hardware load.

5. Profiling & Monitoring

- **Online Profiler:** The system must maintain a running "Weight" (Moving Average + Standard Deviation) of how long each model/batch takes to run.
- **Initial Profile:** The system starts with a `profile.cfg` containing baseline execution times.

- **System Stats:** Continuous logging of CPU/RAM utilization and "Goodput" (successful inferences per second).

6. Performance & Acceptance Criteria

- **Robustness:** The scheduler must handle process failures without crashing. If a SHARK instance fails, the port and cores must be recovered.
- **Concurrency:** Must demonstrate multiple *different* models running at the same time on different ports.
- **SLO Attainment:** In testing (Poisson arrival rate), the SLO-aware algorithms (LST/EDF) must show a higher "Attainment Ratio" than FCFS.
- **Logging:** A comprehensive CSV log (arrival_time, start_time, finish_time, SLO_met_bool, model_type, exit_code, SLO violation %age in 5ms, individual layer execution time).

7. Delivery Requirements

- **Modular C++ Code:** Logic must be separated into Config, FileManager, ResourceManager, and Scheduler.
- **Test Suite:** A script to simulate a Poisson arrival of 100+ requests with a configurable mix of job types.
- **Documentation:** Instructions on how to update `profile.cfg` for new models.